



RAPIDMoE: Exploiting Cross-Asymmetry via Adaptive Residual Offloading for Large-Scale MoE Inference

Wenxun Wang
Tsinghua University, BNRist
Beijing, China

wx-wang23@mails.tsinghua.edu.cn

Likai Ma
Tsinghua University
Beijing, China

mlk22@mails.tsinghua.edu.cn

Zongle Huang
Tsinghua University, BNRist
Beijing, China

huangzl23@mails.tsinghua.edu.cn

Chen Tang
Tsinghua University
Beijing, China
tangc20@mails.tsinghua.edu.cn

Yongpan Liu*
Tsinghua University, BNRist
Beijing, China
ypliu@tsinghua.edu.cn

Abstract

The widespread adoption of Mixture-of-Experts (MoE) has created a growing need for deployment on heterogeneous platforms. However, it exposes a fundamental mismatch between the algorithmic demands of large-scale MoE and the disparate characteristics of hardware. Existing CPU-GPU hybrid inference systems fail to resolve this as they either encounter PCIe bandwidth bottlenecks when loading experts to GPUs, or rely heavily on CPU computation. Consequently, this leads to low resource utilization and inevitable violations of fixed latency budgets as parameters scale.

In this paper, we identify and exploit Cross-Asymmetry—a structural alignment between the algorithmic workload skew of MoE routing and the physical disparity of heterogeneous hardware. To this end, we introduce RAPIDMoE, a residual offloading system for efficient large-scale MoE inference. We propose how RAPIDMoE leverages a residual-split framework to enable offloading paradigm shift from expert-level to bit-level, which unfolds across three key dimensions: (1) data representation, enabling compact and decoupled storage; (2) routing strategy, partitioning computation into dual paths aligned with hardware capabilities; (3) execution parallelism, scheduling a balanced storage-compute workload across devices. We further employ a novel Unified Multi-Level Importance Arbitration to adaptively adjust the critical expert set at runtime, ensuring the accuracy-latency Pareto frontier. These innovations exploit inherent cross-asymmetry, fundamentally breaking the algorithm-hardware misalignment. Experimental results show that RAPIDMoE achieves

up to 3.5× speedup in decoding and 2.1× speedup in prefill compared to state-of-the-art (SOTA) offloading systems.

CCS Concepts: • Computing methodologies → Machine learning; Artificial intelligence; • Computer systems organization → Heterogeneous (hybrid) systems.

Keywords: MoE, Offloading, Algorithm-System Co-design, LLM inference

ACM Reference Format:

Wenxun Wang, Likai Ma, Zongle Huang, Chen Tang, and Yongpan Liu. 2027. RAPIDMoE: Exploiting Cross-Asymmetry via Adaptive Residual Offloading for Large-Scale MoE Inference. In *22nd European Conference on Computer Systems (EuroSys '27)*, April 19–23, 2027, Rabat, Morocco. ACM, New York, NY, USA, 17 pages. <https://doi.org/10.1145/3842654.3848583>

1 Introduction

The advancement of MoEs has spurred their adoption [4, 39, 44, 54], achieving advanced capabilities with large-scale models such as DeepSeek-V3 [12] and Qwen3-235B [50]. This evolution is driving a shift toward the deployment of large-scale MoEs on single-node heterogeneous platforms [28]. As shown in Figure 1, cloud clusters provide capacity but compromise privacy and cost-efficiency, whereas edge devices guarantee privacy but lack the memory and throughput for serving large-scale models [19]. Bridging this gap, the workstation—typically equipped with abundant host DRAM and limited GPU resources (e.g., RTX 4090 or A100)—has become a cost-effective deployment alternative. It satisfies privacy and capacity demands while offering potential to maintain strict Time Per Output Token (TPOT) for interactive streaming under low-concurrency (< 10).

However, the parameter scaling of MoEs vastly exceeds GPU VRAM limits. For instance, deploying DeepSeek-V3 requires over 700GB, forcing most weights to be offloaded to host DRAM. To this end, CPU-GPU hybrid inference systems have been developed. Existing solutions typically employ expert-wise offloading and fall into two paradigms (Figure 2 (b)): (1) *GPU-centric approaches* [5, 43], which cache a subset

*Corresponding Author.



This work is licensed under a Creative Commons Attribution 4.0 International License.

EuroSys '27, Rabat, Morocco

© 2027 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2971-3/2027/04

<https://doi.org/10.1145/3842654.3848583>

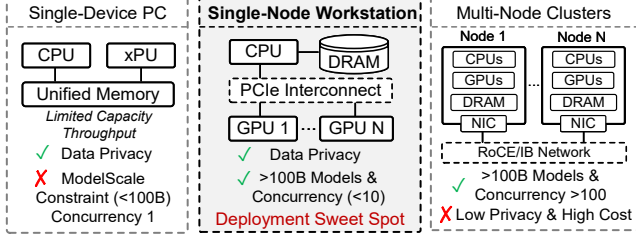


Figure 1. Comparison of deployment scenarios between single-device PC, multi-node clusters and single-node workstation.

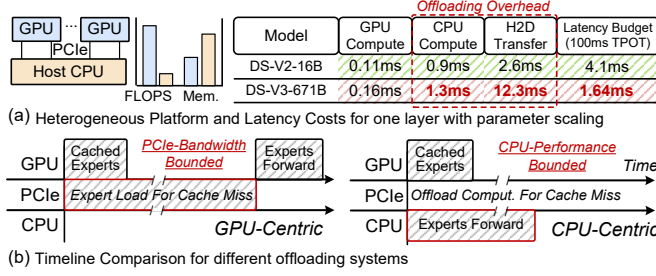


Figure 2. (a) Scaling of MoE offloading overhead against fixed latency constraints on heterogeneous platforms. (b) Timeline illustrating the suboptimal utilization of offloading systems to meet latency demands.

of experts in GPUs and prefetch missing ones via PCIe; and (2) *CPU-centric approaches* [26, 28], which shift MoE computation to CPUs for absent experts to bypass I/O bottlenecks.

Despite enabling large-scale MoE execution, these systems hit an offloading dilemma. This manifests in two dimensions: latency and efficiency. As parameters grow, the overhead of CPU offloading—including PCIe transfers and CPU computation—scales correspondingly. This growth directly conflicts with the strict and constant latency budget (e.g., 100ms TPOT), leading to latency violations. Meanwhile, GPUs remain underutilized as the pipeline stalls waiting for CPU computation or I/O, resulting in low overall efficiency. The root cause lies in a fundamental **algorithmic-hardware misalignment**. Heterogeneous platforms exhibit pronounced resource asymmetry: GPUs offer high throughput but limited memory, whereas CPUs provide abundant memory capacity but low throughput. Ideally, storage-heavy workloads should reside on CPUs and compute-intensive workloads on GPUs. However, the scaling latency of transferring gigabytes of expert weights between devices forces tight coupling between computation and storage: an expert must be computed where its weights reside. This binding fails to align with hardware, preventing efficient MoE inference.

We identify the **Cross-Asymmetry** in both algorithmic demands and hardware characteristics as the key to break

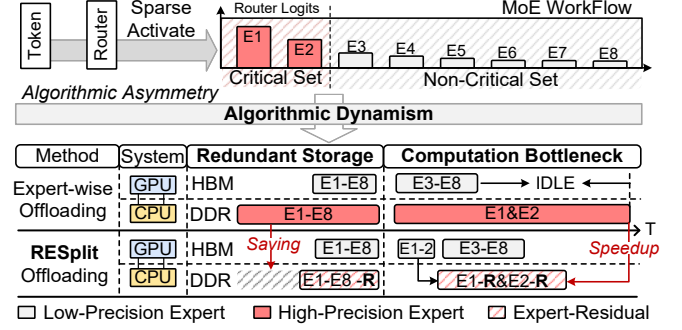


Figure 3. Illustration of algorithmic asymmetry and comparison between expert-wise and RESplit offloading in leveraging asymmetry.

the deadlock. On the algorithmic side, MoE routing exhibit skewed importance distributions [35, 42] (Figure 3): a few critical experts demand high-precision storage to maintain model quality, but incurs low floating-point operations (FLOPS) with limited number. Conversely, the remaining non-critical experts tolerate low precision but dominate the overall computation. This algorithmic asymmetry naturally aligns with the hardware disparity: CPUs handle the storage-intensive, low-FLOP critical experts, while GPUs excel at the storage-light, high-FLOP non-critical ones.

Exploiting the cross-asymmetry exposes three fundamental challenges in system design. First, expert importance varies across different tokens, layers and inference phases, requiring a runtime arbitration to balance accuracy against latency for **algorithmic dynamism**. Second, the dynamic expert importance conflicts with static expert-wise storage, inevitably leading to **redundant storage** as systems maintain dual copies of high/low-precision across CPUs and GPUs. Third, the hardware asymmetry renders the expert-wise scheduling a fundamental **computation bottleneck**. As even a single expert on CPU becomes the straggler (Figure 6), expert-wise offloading fails to achieve efficient utilization.

To address this challenge, we present RAPIDMoE to exploit Cross-Asymmetry for efficient large-scale MoE inference. To tackle algorithmic dynamism, we uncover the inherent multi-level importance divergence in MoE routing and develop a novel **Unified Multi-Level Importance Arbitration (UMIA)**, which identifies critical experts by integrating *local* routing confidence, *spatial* layer-wise and *temporal* phase-wise importance, achieving an effective trade-off between system latency and accuracy. To avoid storage and computation dilemmas, We introduce an offloading paradigm shift from expert-level to bit-level through a systematic **residual-split (RESplit)** framework. At the **data representation**, RESplit proposes a residual decomposition to split weights at the bit level, enabling a compact storage and a hardware-aware mapping: memory-intensive residuals W^R are offloaded to CPUs, while quantized components W^Q remain on GPUs.

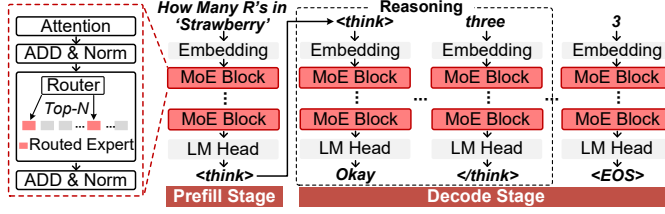


Figure 4. Inference of MoE LLMs with reasoning.

At the **routing strategy**, split expert routing is developed to mirror this decomposition in computation. The top- k routed experts are divided into a small set of critical experts (top- r) and the remaining top- $(k-r)$ non-critical experts. Non-critical experts activate only W^Q , while critical experts utilize both W^Q and W^R to ensure high-precision computation. At the **execution layer**, RESplit expert parallelism provides fine-grained, hardware-aware workload assignments. Storage-wise, lightweight W^Q is assigned to GPUs while storage-intensive W^R is offloaded to CPUs, avoiding redundant storage. Computation-wise, high-FLOP non-critical experts are handled by GPUs using W^Q while CPUs refine a minority of critical experts via W^R , breaking computation bottlenecks. Together, these techniques enable RAPIDMoE to achieve optimal latency and efficiency while maintaining model accuracy. The key innovations are summarized as follows:

- We formalize cross-asymmetry as the foundational principle for large-scale MoE inference on heterogeneous platforms: the algorithm-hardware alignment between skewed MoE routing and resource disparity.
- We introduce UMIA to unify local, spatial, and temporal-wise importance for adaptive discrimination, enabling importance-aware workload partitioning.
- We propose systematic RESplit methods spanning routing strategy, data representation, and parallelism design, enabling hardware-aware workload partitioning across storage and computation.
- We implement and evaluate RAPIDMoE on typical heterogeneous platforms with DeepSeek-671B and Qwen3-235B, achieving up to 3.5 $\times$ speedup over SOTA systems.

2 Background

2.1 Mixture of Experts and Reasoning Models

MoE [9, 30, 39, 40] introduces sparsity by employing multiple expert feed-forward layers and a dynamic gating function to route each input token to the most relevant subset of experts, as shown in Figure 4. For an input token x , the gating function $g(x)$ computes distribution logits via a linear projection and softmax normalization. Sparsity is enforced by activating the top- k experts with the highest logits. The final output combines the routed experts: $\sum_{i=1}^N g_i E_i(x)$, where $E_i(x)$ denotes the i -th expert output, and g_i is its gating weight. By

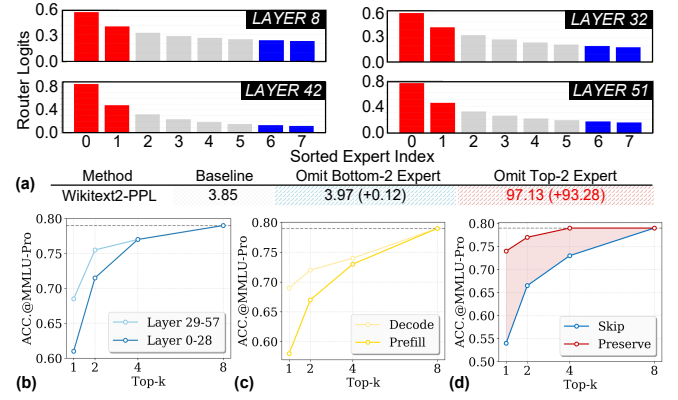


Figure 5. Algorithmic evaluation on DeepSeek-V3: (a) Local divergence highlighted by router distribution and perplexity on Wikitext-2. (b-c) Spatial and temporal divergence evaluated by applying various top- k routed experts. (d) Comparison of expert skipping and expert preserving.

activating a subset of experts, MoE maintains computation efficiency, for example, DeepSeek-R1 [11] activates 37B of its 671B parameters ($k=8$). The rise of reasoning models further enhances their capabilities [47, 50], improving model accuracy by test-time scaling [27, 38].

Despite the benefit, MoE models have far more parameters than dense models of comparable performance. For instance, while achieving similar accuracy, DeepSeek-R1 contains 266B more parameters compared to LLaMA-3.1-405B [23], translating to an extra 532GB of memory requirement. It leads to great costs as all parameters must remain in limited GPU VRAM despite only a small subset being activated. Meanwhile, the enhanced performance of reasoning comes at the cost of increased decoding lengths. DeepSeek-R1 achieves 96% accuracy on MATH-500 but requires an average 9363 output tokens, whereas non-reasoning models average fewer than 2048 tokens on the same benchmark. This extended decoding time dominates end-to-end latency [20], creating the latency challenge for large-scale MoE serving.

2.2 CPU/GPU Hybrid Inference for MoEs

To address the deployment challenges, existing systems employ CPU/GPU hybrid inference with expert-wise offloading, which can be categorized into GPU-centric and CPU-centric offloading based on computation device.

Firstly, GPU-centric offloading systems execute MoE layers on GPUs [5, 43]. When the required experts are absent, they are transferred from CPU via PCIe connection. Given the limited GPU memory capacity, current works propose Caching and Prefetching. Caching maintains a subset of experts in GPUs while offloading others to CPUs. The policy of cache is managed by identifying important experts offline [26], tracking hot experts online and adopting least frequently used for replacement [43, 53]. Prefetching uses

gating logits to predict routed experts in subsequent layers and overlap the expert loading with GPU computation of current layer [43, 49]. Despite addressing capacity concerns, **these techniques fail to meet the low-latency requirements due to the reliance on CPU-GPU expert loading.** Prefetching attempts to hide loading latency by overlapping, yet the loading time remains on critical path. Caching suffers from high miss rates (e.g., 70% [26]) under limited GPU memory, which in turn forces frequent loading. Limited PCIe bandwidth results in unacceptable latency. As shown in Figure 2, transferring 8 experts for a single DeepSeek-V3 layer over PCIe 4.0 takes 12.3 ms—more than 7× the 1.64 ms latency budget required to achieve a 100 ms TPOT target.

Secondly, CPU-centric systems offload MoE layer computation to CPUs [22, 26, 28]. These works design CPU-friendly kernels with SIMD instruction sets [22], and utilize CUDA graphs to reduce inference latency [28]. However, **this design creates an imbalance workloads, as most workloads are shifted to CPUs while leaving GPUs underutilized.** As detailed in Figure 6 (c), even when only half the per-layer workload is executed on the CPU (i.e., 4 out of 8 experts), GPU utilization remains limited at only 10–20%. Consequently, it degrades resource utilization and represents a mismatch with hardware disparity, where GPUs offer orders-of-magnitude higher throughput. HybriMoE [56] attempts to dynamically schedule partial workloads to GPUs via overlapping CPU-to-GPU weight transfers with computation. However, this benefit diminishes for large-scale MoEs; the latency of transferring gigabyte-scale experts cannot be amortized by compute overlap (Figure 2a).

3 Motivation

We identify the inherent algorithm-hardware cross-asymmetry as the key to resolving the mismatch.

3.1 Uncovering Algorithmic Asymmetry

Observation 1: Multi-Level Importance Divergence.

Local Divergence: The gate router outputs show skewed distributions where a small subset of experts dominates the outputs (Figure 5 (a)). Empirically, omitting the two lowest-scoring experts increases perplexity by merely 0.12, whereas dropping the top-2 experts severely degrades outputs by 93.28. This reveals that *local* importance, captured by routing logits g , is concentrated in a sparse subset with only a few critical experts requiring high-precision.

Spatial Divergence: Router logits only capture input-dependent importance within a single layer, ignoring the varied *spatial* impact across layers. Reducing top- k routing in shallower layers incurs larger quality degradation than in deeper layers as shown in Figure 5 (b). This stems from error amplification: errors introduced in early layers propagate through subsequent layers, causing potential explosion [2]. It suggests that the optimal size of critical set should vary across layers.

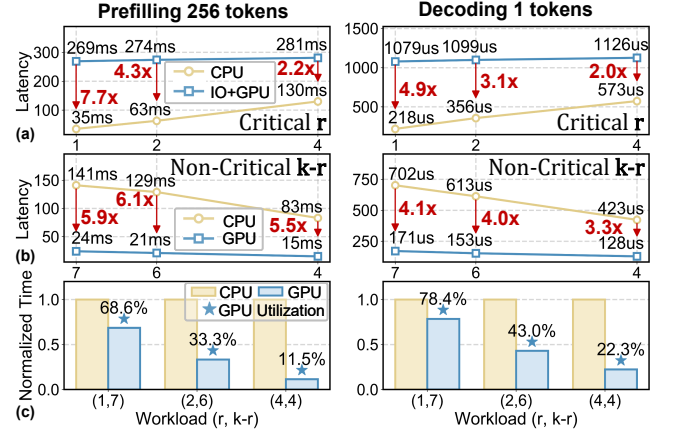


Figure 6. (a-b) Latency of critical (INT4) and non-critical (INT2) experts. Critical experts favor CPUs to bypass PCIe I/O bottlenecks, while non-critical experts maximize throughput on GPUs. (c) Timeline analysis of expert-wise offloading. Even with an ideal mapping (critical on CPUs, non-critical on GPUs), it incurs sub-optimal GPU utilization.

Temporal Divergence: The nature of LLM inference that operates in prefill and decode phase sequentially introduces a *temporal* divergence [6]. Reducing top- k during prefilling degrades accuracy substantially more than during decoding (Figure 5 (c)), due to the foundational role of prefilling in constructing high-quality KV caches as semantic contexts for subsequent decoding generation. Thus, system must prioritize higher accuracy during prefilling compared to decoding.

These divergences motivate a unified arbitration framework as detailed in Section 4.3.

3.2 Leveraging Algorithmic Asymmetry

Observation 2: Expert Preserving over Skipping.

A naive approach to exploit this asymmetry is *expert skipping*—completely omitting non-critical experts [3, 35]. However, aggressive skipping reduces model generalization capabilities to diverse input workloads. As shown in Figure 5 (d), skipping the non-critical experts even at $k = 4$ causes severe performance loss. In contrast, *expert preserving*, where non-critical experts are preserved in low precision, restores most of the lost generation quality. This suggests that **while high-score experts are critical for preserving quality, preserving partial information for low-score experts is still essential for robustness.** This motivates our split expert routing to fully realize the theoretical efficiency for *expert preserving* as detailed in Section 3.2. Specifically, routed experts are divided into two sets with complementary resource demands: (1) critical top- r experts, requiring high-precision storage but with less computation workload, and (2) the remaining top- $(k - r)$ experts, which involve more workload but tolerate lower precision.

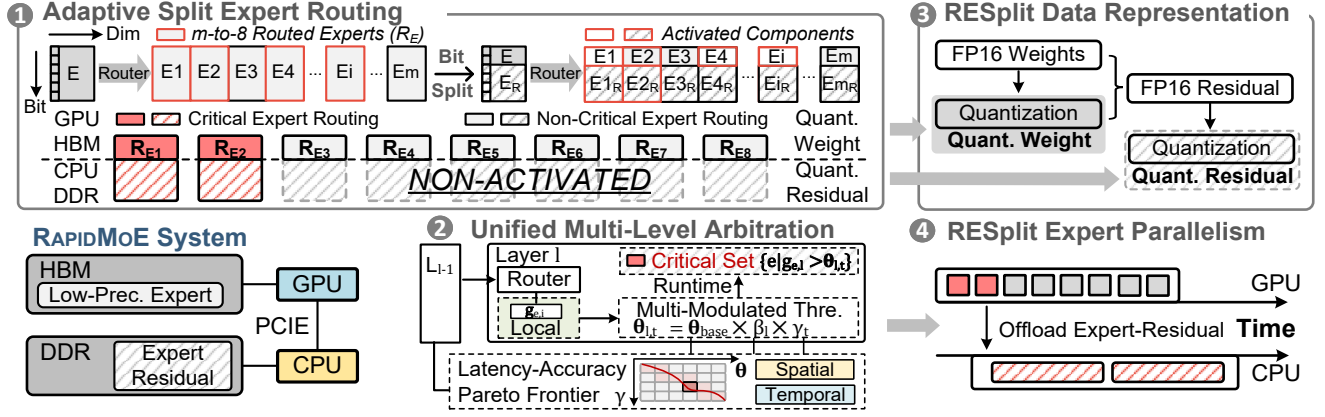


Figure 7. Overview of RAPIDMoE. UMIA splits routed experts runtime, while RESplit enables efficient storage and execution.

3.3 Aligning Cross-Asymmetry

Cross-Asymmetry Alignment. Modern heterogeneous platforms inherently exhibit a compute-memory disparity [15, 41]. For LLM workloads, GPUs deliver immense computation throughput (e.g., 312 TFLOPS for FP16 on A100) but are bounded by HBM capacity (e.g., 80GB). Conversely, host CPUs offer massive memory capacity (up to 1TB DDR) but possess drastically lower compute throughput. Such disparity demands hardware-aware workload partitioning. Through our profiling at Figure 6, we identify the cross-asymmetry as the resource demands of MoE experts mirror this hardware disparity: First, critical experts demand high precision (e.g., INT4) but constitute minimal FLOPs ($r < k - r$). Executing them on the high-capacity CPU bypasses severe PCIe I/O bottlenecks, yielding lower latency. Second, non-critical experts tolerate lightweight quantization (e.g., INT2) but dominate the computation workload. Their reduced footprint fits within GPU HBM, fully unleashing the GPU’s throughput. Under parallel execution, latency is bounded by $\max(t_{cpu}, t_{gpu})$. This algorithmic asymmetry naturally compensates for the hardware disparity, effectively closing the execution time gap between t_{cpu} and t_{gpu} and maximizing GPU utilization.

While cross-asymmetry theoretically balances the workload, we observe a fundamental flaw of existing expert-level scheduling paradigm in managing cross-asymmetry:

Observation 3: Bottlenecks of Expert-Level Scheduling.

Redundant Storage. As the importance varies dynamically, experts cannot be statically pinned to a single precision level (i.e., high or low) for storage. Under the expert-level paradigm, the system is forced to maintain dual copies of weights—both high and low precision—for every expert to cover all routing possibilities, inevitably leading to redundant storage. This memory overhead is non-trivial for large-scale models; for instance, DeepSeek-R1 incurs an additional 170GB memory simply to store 2-bit low-precision replicas.

Algorithm 1: Split Expert Routing Mechanism

Input: x : input token
 W^H : high-precision experts weights
 W^L : low-precision experts weights
Output: y : weighted sum
 $g \leftarrow \text{TopK}(\text{Router}(x))$
 $e_c, e_n \leftarrow \text{Unified Multi-Level Importance Arbitration}(g)$
 $y \leftarrow 0$
foreach $e \in e_c$ **do**
 $y \leftarrow y + g_i \cdot e(x, W^H)$ // Critical Experts
foreach $e \in e_n$ **do**
 $y \leftarrow y + g_i \cdot e(x, W^L)$ // Non-Critical Experts
return y

Computation Bottleneck. As shown in Figure 6 (c), expert-level scheduling also imposes a fundamental computation bottleneck. Since experts are treated as indivisible atomic units, systems experience GPU bubbles even under an ideal mapping (critical experts on CPUs, non-critical on GPUs) with a minimal CPU workload (e.g., $r = 1$).

This reveals that aligning cross-asymmetry requires fine-grained scheduling to overlap execution without redundant storage, which drives the paradigm shift to the RESplit mechanism (Secs. 4.4 and 4.5).

4 RAPIDMoE System

4.1 Design Overview

We propose RAPIDMoE, a system for efficient large-scale MoE inference on heterogeneous platforms. As illustrated in Figure 7, RAPIDMoE adopts a co-design approach. At the algorithmic level, we propose split expert routing to exploit asymmetry via preserving (Observation 2), which decomposes the top- k routing into top- r critical and top- $(k - r)$ non-critical sets. Building upon Observation 1, we introduce UMIA to adjust the critical expert set at runtime, ensuring

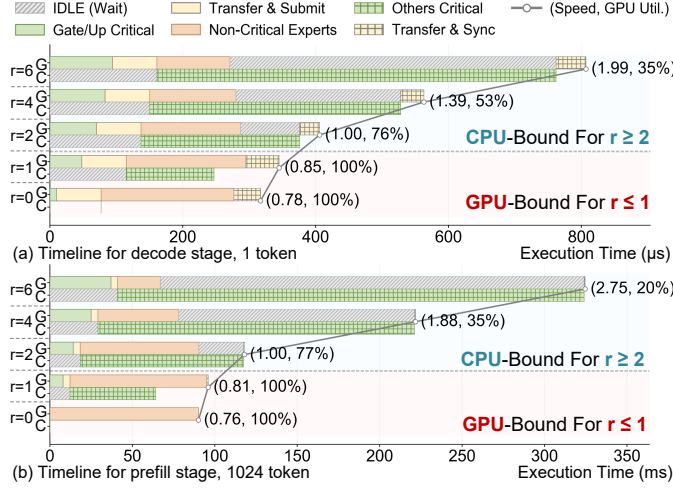


Figure 8. Per-layer RESplit timelines for DeepSeek-V3 at different critical-set sizes r on P1 (Table 3), with latency normalized to $r = 2$: (a) single-token decode and (b) 1024-token prefill. The execution bottleneck shifts at $r = 2$.

the accuracy-latency Pareto frontier. Informed by Observation 3, we adopt RESplit representation and parallelism to address the storage and computation bottlenecks. By anchoring low-precision W^Q on GPUs and offloading residuals W^R to CPUs, GPUs deliver computation for non-critical experts utilizing W^Q , while CPUs concurrently handle the residual for critical experts with W^R .

4.2 Split Expert Routing Mechanism

Based on the standard top- k MoE routing [9, 50], we propose the split expert routing mechanism as depicted in Algorithm 1. It divides routed experts into critical and non-critical sets based on router logits, selecting the top- r experts as critical. For critical experts, we retrieve high-precision weights W^H to ensure accuracy, while fetching low-precision weights W^L to handle non-critical experts. Since non-critical experts produce smaller output weights g_i , this design preserves accuracy while reducing computation cost. Notably, it exploits the inherent router logits in MoE, avoiding any task-specific calibration or retraining.

Asymmetric Latency Scaling. For heterogeneous systems, the latency of an MoE layer can be formulated as:

$$T_{sys} = t_{sync} + \max\left(\frac{\text{Load}_{cpu}}{\text{Perf}_{cpu}}, \frac{\text{Load}_{gpu}}{\text{Perf}_{gpu}}\right) \quad (1)$$

s.t. $\text{Load}_{cpu} \propto r$, $\text{Load}_{gpu} \propto k - r$

where t_{sync} captures CPU-GPU synchronization overheads. The choice of r represents a trade-off between model accuracy and system latency. To optimally configure r , we conduct a latency profiling at Figure 8, executing the proposed pipeline in Sec. 4.5. The profiling reveals key system behaviors. First, the system experiences a bottleneck shift dictated

Table 1. Summary of UMIA parameters.

Parameter	Description
$g_e^{(l)}$	Local importance of expert e in layer l
$\beta^{(l)}$	Spatial importance of layer l
$\gamma^{(t)}$	Temporal importance at phase t
$e_c^{(l,t)}, r^{(l,t)}$	Critical expert set and its size at layer l , phase t
Θ, α	Global base threshold and phase scaling factor
$\mathcal{D}, \mathcal{C}, \mathcal{A}$	Calibration set and candidate sets for Θ and α
N, K	Sizes of $\mathcal{D}$ and candidate sets $\mathcal{C}, \mathcal{A}$
$\hat{T}_{p,d}, S_{p,d}$	Estimated prefill/decode latency and their SLOs

by $\text{Perf}_{gpu}/\text{Perf}_{cpu}$. The transition point r^* , at which CPU and GPU execution times align ($\frac{r}{\text{Perf}_{cpu}} \approx \frac{k-r}{\text{Perf}_{gpu}}$), is inherently skewed (e.g., $r^* \in [1, 2]$). Consequently, the system is GPU-bound when $r \leq r^*$ and CPU-bound when $r > r^*$. Second, this shift induces an *asymmetric scaling behavior*: in the CPU-bound region, reducing r lowers latency by alleviating CPU workload, whereas in the GPU-bound regime, further reductions yield diminishing returns. Third, the synchronization overhead exhibits *phase-dependency*. While negligible during the compute-intensive prefill stage, it accounts for $\sim 20\%$ of the decode latency, as the short latency of decoding makes a 10-100 μs synchronization barrier non-negligible.

Based on these empirical insights, we formulate a simplified relative latency estimator. Let $R(r) = T_{sys}(r)/T_{sys}(r^*)$ denote the relative latency for prefill (R_p) and decode (R_d). We introduce a penalty factor η to capture the marginal reduction within the GPU-bound regime, and use σ for synchronization overhead:

$$R_p(r) = \begin{cases} 1 - \eta(r^* - r) & r < r^* \\ \frac{r}{r^*} & r \geq r^* \end{cases}, R_d(r) = \sigma + (1 - \sigma)R_p(r) \quad (2)$$

System Implication. This asymmetric and phase-dependent relationship reveals that statically constraining the expert number r fails to reflect system latency gain. It directly motivates the system-aware latency constraint formulation in UMIA (Section 4.3). In our implementation, we build the estimator with $(\sigma, \eta, r^*) = (0.2, 0.15, 2)$ based on our profiling. While these parameters can be easily calibrated via a one-time micro-benchmark for different platforms, the structural asymmetry of $R(r)$ remains robust to variations across platforms, as it is designed to capture the relative latency scaling trends rather than to provide absolute timing. This robustness is further validated in the cross-platform evaluation in Sec. 6 (P1 and P2 in Table 3).

4.3 Unified Multi-Level Importance Arbitration

We introduce Unified Multi-Level Importance Arbitration (UMIA), acting as a training-free, lightweight engine. Offline,

Algorithm 2: Hetero-Aware Frontier Search

Input: Calibration set $\mathcal{D}$, SLO constraints S_p, S_d , candidate sets $\mathbf{C}, \mathbf{A}$ (ascending order).

Output: Parameters (Θ^*, α^*) .

$\mathcal{F} \leftarrow \emptyset$;

foreach prompt $x \in \mathcal{D}$ **do**

$\text{FOUND} \leftarrow \text{false}$;

foreach $\Theta \in \mathbf{C}$ **do**

 // Stage 1: Prefill forward

$r^p \leftarrow \text{PREFILL}(x; \Theta)$;

$\hat{T}_p \leftarrow T_{sys}(r^p)$;

if $\hat{T}_p > S_p$ **then continue**;

 // Stage 2: Decode forward

foreach $\alpha \in \mathbf{A}$ **do**

 // M Decode Steps

$r^d \leftarrow \frac{1}{M} \sum_{i=1}^M \text{DECODE}(x; \Theta, \alpha, i)$;

$\hat{T}_d \leftarrow T_{sys}(r^d)$;

if $\hat{T}_d > S_d$ **then continue**;

 // Add Frontier Candidate

$\mathcal{F}, \text{FOUND} \leftarrow \mathcal{F} \cup \{(\Theta, \alpha)\}$, **true**;

 // Early stop: $\mathcal{L}$ increases with α

break;

 // Early stop: $\mathcal{L}$ increases with Θ

if FOUND **then break**;

// Evaluate frontier candidates

foreach $(\Theta, \alpha) \in \mathcal{F}$ **do**

$\bar{\mathcal{L}}_p(\Theta) \leftarrow \frac{1}{|\mathcal{D}|} \sum_{x \in \mathcal{D}} \text{PREFILLEVAL}(x; \Theta)$;

$\bar{\mathcal{L}}_d(\Theta, \alpha) \leftarrow \frac{1}{|\mathcal{D}|M} \sum_{x \in \mathcal{D}} \sum_{i=1}^M \text{DECODEVAL}(x; \Theta, \alpha, i)$;

return $(\Theta^*, \alpha^*) \leftarrow \text{argmin}_{(\Theta, \alpha) \in \mathcal{F}} (\omega_p \bar{\mathcal{L}}_p + \omega_d \bar{\mathcal{L}}_d)$;

UMIA calibrates deployment parameters (Θ, α, β) under latency target S . At runtime, it uses the MoE router score $\mathbf{g}$ to adaptively determine the optimal critical set size r .

Multi-Level Modulation. Instead of employing a fixed r or static thresholding, UMIA computes a multi-modulated threshold (MMT) for each layer and execution phase. For a given MoE layer l at phase t , UMIA defines the MMT $\Theta^{(l,t)}$ as a multiplicatively separable modulation:

$$\Theta^{(l,t)} = \underbrace{\Theta}_{\text{Base}} \times \underbrace{\beta^{(l)}}_{\text{Spatial}} \times \underbrace{\gamma^{(t)}}_{\text{Temporal}} \quad (3)$$

where Θ is the global base threshold capturing inherent model redundancy, $\beta^{(l)}$ is the layer-wise *spatial* modulation coefficient, and γ_t is the phase-wise *temporal* coefficient. Combined with *local* importance $\mathbf{g}$, UMIA constructs the critical set $\mathbf{e}_c^{(l,t)}(x)$ for inputs x by the gating:

$$\mathbf{e}_c^{(l,t)}(x) = \left\{ e \mid \mathbf{g}_e^{(l)}(x) \geq \Theta^{(l,t)} \right\}, \quad r^{(l,t)} = |\mathbf{e}_c^{(l,t)}| \quad (4)$$

The remaining $k - r^{(l,t)}$ experts form the non-critical set. Here, Θ controls gate selectivity: a larger Θ imposes a stricter gating criterion, admitting fewer critical experts and thus reducing the critical-set size r . Table 1 summarizes the key UMIA parameters and notation. Specifically, it unifies importance across:

- **Local Modulator $\mathbf{g}$:** Router logits of MoE layer l naturally capture intra-layer expert importance for given tokens, normalized by softmax such that $\mathbf{g} \in [0, 1]$.
- **Spatial Modulator β :** To quantify layer-wise asymmetry, β is derived offline via Kullback-Leibler (KL) divergence as $\beta_l = (\frac{1}{N} \sum_{i=1}^N D_{KL}(P_i \parallel P_i^{(l)}))^{-1}$, where P_i is the BF16 reference distribution and $P_i^{(l)}$ is the quantized counterpart using low-precision experts at layer l . A larger β therefore denotes a less sensitive layer.
- **Temporal Modulator γ :** UMIA models it as a binary-state parameter corresponding to the inference phase:

$$\gamma_t = \begin{cases} 1, & \text{if } t \text{ is Prefill} \\ \alpha, & \text{if } t \text{ is Decode} \end{cases} \quad (5)$$

Based on the analysis at Sec. 3.1, we constrain $\alpha > 1$ to ensure that prefilling utilizes a larger set of $\mathbf{e}_c$ compared to decoding to maintain KV cache quality.

Hetero-aware Frontier Search. Given the router scores $\mathbf{g}$ and KL-calibrated β , UMIA configures (Θ, α) via an offline *Hetero-Aware Frontier Search* to balance system latency and model accuracy. While prior fixed-sparsity calibration [3, 35] (e.g., globally skipping 75% of experts) fails to capture the asymmetric scaling and phase-dependency feature as analyzed in Sec. 4.2, UMIA replaces sparsity constraints with latency constraints S :

$$\begin{aligned} \min_{\Theta \in \mathbf{C}, \alpha \in \mathbf{A}} \quad & \bar{\mathcal{L}}(\Theta, \alpha) = \omega_p \bar{\mathcal{L}}_p(\Theta) + \omega_d \bar{\mathcal{L}}_d(\Theta, \alpha), \\ \text{s.t.} \quad & \hat{T}_p(\Theta) \leq S_p, \quad \hat{T}_d(\Theta, \alpha) \leq S_d, \end{aligned} \quad (6)$$

(Θ, α) determine the critical-set sizes r via Eq. 4. These sizes are then mapped to estimated execution latency $\hat{T}$ using the latency estimator $R(r) \times T_{sys}(2)$ in Eq. 2. $\bar{\mathcal{L}}_{p,d}$ denote the average KL-divergence losses over the calibration set $\mathcal{D}$ for prefill and decode, respectively. The constraints S explicitly bound execution latency (e.g., an SLO equivalent to $T_{sys}(2)$), rather than impose a fixed number of low-precision experts.

UMIA solves the optimization via a two-stage pipeline in Algorithm 2. Crucially, increasing (Θ, α) reduces the number of high-precision experts r , thereby lowering latency (Figure 8) and increasing quantization loss, yielding a monotone accuracy-latency trade-off. UMIA exploits this monotonicity by enumerating candidates in ascending order with early stopping mechanism. Specifically, for each prompt $x \in \mathcal{D}$, Stage 1 scans $\mathbf{C}$ in ascending order, skipping SLO-violated candidates before costly decode evaluations. For each prefill-feasible Θ , Stage 2 similarly finds the first decode-feasible α . Since latency decreases while KL loss increases with Θ

Algorithm 3: Residual-Split Data Representation

Function $RESplit(W, b_q, b_r)$
Input: W : FP16 weight, b_q : bit-width, b_r : residual bit-width
Output: W^Q : quantized weight, W^R : quantized residual
 $W^Q \leftarrow \text{Quantize}(W, b_q)$
 $W^R \leftarrow \text{Quantize}(W - W^Q, b_r)$; // Residual
return (W^Q, W^R)

Function $ExpertRESplit(W_{gate}, W_{up}, W_{down}, b_q, b_r)$
Input: $\{W_{gate}, W_{up}, W_{down}\}$: Expert weights
Output: Quantized expert weights
 // Gate/Up with RESplit representation
 $(W_{gate}^Q, W_{gate}^R) \leftarrow RESplit(W_{gate}, b_q, b_r)$
 $(W_{up}^Q, W_{up}^R) \leftarrow RESplit(W_{up}, b_q, b_r)$
 // Down with standard representation
 $W_{down}^L \leftarrow \text{Quantize}(W_{down}, b_q)$
 $W_{down}^H \leftarrow \text{Quantize}(W_{down}, b_q + b_r)$
return (W^H, W^L)

and α , the first feasible pair (Θ, α) minimizes loss within the feasible region, allowing early stopping to terminate the remaining search for the current prompt. This yields one candidate pair for each x , and the resulting pairs collectively form $\mathcal{F}$. Finally, Alg. 2 evaluates each pair in $\mathcal{F}$ over the full calibration set and selects the one minimizing the prompt-averaged weighted loss in Eq. 6.

Calibration Overhead. As a *once-for-all* offline optimization, UMIA incurs a modest calibration cost. Profiling β requires $\mathcal{O}(LN)$ forward passes (about 30 minutes for $N = 16$). We configure the search range of (Θ, α) such that, when S_p and S_d permit similar critical-set sizes ($\Delta r \leq 2$), a jointly feasible pair is found within the first three prefill-feasible Θ . Frontier construction and final evaluation therefore require $\mathcal{O}(NK)$ and $\mathcal{O}(N^2)$ generations, respectively, for an overall complexity of $\mathcal{O}(NK)$ since $K \gg N$. With $N, K = 16, 100$, each generation (1 prefill and 100 decode steps) takes about 10 seconds on a 671B MoE, and the full search completes in about 5 hours—a negligible one-time cost for deployment.

Runtime Arbitration. At deployment, UMIA applies the pre-computed (Θ, β, α) via lightweight gating (Eq. 4), which can be fused into the MoE inline gating kernel with negligible overhead. For multi-token scenarios (i.e., prefill stage and decoding with batch>1), we take the token-averaged g for UMIA decision-making. The MMT dynamically partitions workload between CPUs and GPUs, ensuring the system operates at a favorable accuracy–latency frontier.

4.4 Residual-Split Data Representation

As revealed in Observation 3, the varying expert importance introduces notable memory overhead by dual-weight storage.

For instance, DeepSeek-R1 incurs an additional 170GB memory demand even with 2-bit W^L . Therefore, we implement RESplit data representation to eliminate redundant storage.

Residual-Split Representation. We employ a two-stage split representation as detailed in Algorithm 3. First, weights are quantized to low precision as W^Q . The residual $W - W^Q$ is then quantized as W^R . This two-stage approach preserves weight information and reduces quantization error through residual compensation. High and low-precision weights can be represented by $W^H = W^Q + W^R$, $W^L = W^Q$. While avoiding dual-copy storage, it also enables efficient parallel computation in CPU-GPU systems through linear decomposability: $XW^H = XW^Q(\text{GPU}) + XW^R(\text{CPU})$.

GLU Architecture Adoption. For linear layers, residual quantization enables parallel computation through the decomposition $XW^Q + XW^R$. However, the implementation becomes challenging due to gated linear unit (GLU) structure. The GLU computation $O = W_{down}(\text{SiLU}(W_{gate}x \odot W_{up}x))$ introduces nonlinearity that disrupts parallelization for *down_proj* layer. Specifically, when W^Q and W^R are stored separately, *gate_proj* and *up_proj* maintain linearity and can be decomposed as $XW_{gate/up}^Q + XW_{gate/up}^R$. However, the non-linear Swish function before *down_proj* requires full intermediate results, forcing activation gathering and synchronization after *gate_proj* and *up_proj*. Such frequent CPU-GPU synchronization fragments kernels, increases launch overhead and undermines CUDA graph optimization. To balance computation and storage efficiency, we apply residual quantization only to *gate_proj* and *up_proj*, while employing dual-quantization for *down_proj*. Figure 9 shows an execution example of critical expert forwarding, with CPU-GPU synchronization only at the start and end of the operator. The detailed computation flow is discussed in Sec. 4.5.

4.5 Residual-Split Expert Parallelism

Expert-wise offloading that assigns high-precision experts to GPUs and low-precision ones to CPUs inevitably leads to GPU bubbles as demonstrated in Figure 6. We therefore develop RESplit parallelism to fully exploit GPU throughput.

Memory Allocation. RAPIDMoE employs a compact, structured memory allocation where expert weights are stored sequentially as low-precision W^Q followed by residuals W^R . At initialization, W^Q and W^R are stored in CPU memory, with W^Q subsequently loaded onto GPUs. This design offers two key advantages: (1) tokens can be processed through low-precision W^Q on GPUs; and (2) CPUs only need to access the lightweight residuals W^R to refine a small set of critical experts. This approach, termed Expert-Residual Offloading, is elaborated in the next paragraph.

Expert-Residual Offloading. Experts follow a Swish-gated linear unit structure: $O = W_{down}(\text{SiLU}(W_{gate}x \odot W_{up}x))$. We decompose the pre-Swish computations as $h_{up/gate} = W_{up/gate}^Q x + W_{up/gate}^R x$, where W^Q and W^R denote low-precision

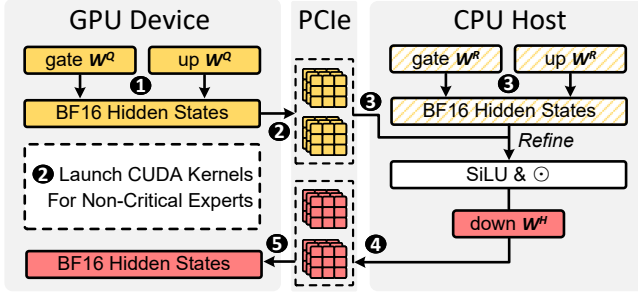


Figure 9. Execution example for critical experts with residual-offloading.

weights and quantization residuals, respectively. With the split layout in RAPIDMoE, this design naturally enables heterogeneous computation:

$$\underbrace{W_{\text{down}}}_{\text{CPU}} \cdot \left(\text{SiLU} \left(\underbrace{W_{\text{gate}}^Q x}_{\text{GPU}} + \underbrace{W_{\text{gate}}^R x}_{\text{CPU}} \right) \odot \left(\underbrace{W_{\text{up}}^Q x}_{\text{GPU}} + \underbrace{W_{\text{up}}^R x}_{\text{CPU}} \right) \right) \quad (7)$$

where GPUs handle the $W_{\text{gate/up}}^Q x$ terms, and CPUs process the residual $W_{\text{gate/up}}^R x$ terms. It further alleviates CPU bottlenecks while preserving computation accuracy.

Computation Flow. Figure 8 shows the detailed RESplit timeline, with corresponding steps marked in Figure 9. For each input token and its routed experts, GPUs first compute the *gate_proj* and *up_proj* of critical experts (**Gate/Up Critical, ①**). After transferring these activations to CPUs (**Transfer&Submit**), CPUs launch computation threads while GPUs concurrently process non-critical experts using low-precision W^Q (**Non-Critical Experts, ②**). In the residual forward pass (**Others Critical**), CPUs access W^R for *gate_proj* and *up_proj*, compute residual hidden states, and merge them with GPU-computed states (**③**). CPU outputs are then generated through Swish activation and high-precision *down_proj* (**④**). The system performs synchronization to derive final outputs from GPUs and CPUs (**Transfer&Sync, ⑤**).

This parallelism benefits both decode and prefill phases by fine-grained scheduling that reserves only critical expert residuals for CPUs. Compared with the naive scheduling pipeline in Figure 6, it achieves higher GPU utilization across various settings of r , improving from 33% to 77% at $r = 2$, and from 69% to 100% at $r = 1$.

5 Implementation Details

We build RAPIDMoE on top of KTransformers [28] framework, which provides a flexible infrastructure for kernel injection and hybrid CPU-GPU execution. Residual-split parallelism is implemented using fine-grained CUDA stream scheduling, enabling the parallel execution across CPUs, GPUs and PCIe transfers. For MoE operations, we design

Table 2. Configuration of evaluated large-scale MoE models.

MoE LLMs	DeepSeek-V3/R1	Qwen3-235B-A22B
#Total Parameters	671B	235B
#Activated Parameters	37B	22B
#Expert Num	256	128
#Routed Expert/top-k	8	8
#Shared Expert	1	0

specialized C++ kernels for CPU expert-residual forwarding and flexible CUDA kernels for GPU residual-offloading. Both are capable of being captured by CUDA graphs to eliminate kernel launch overhead. For non-MoE operations, we utilize the MARLIN kernels [18] for linear layers and the customized flashinfer backend [28, 52] for attention.

Quantization Framework. We adopt the GPT-generated Unified Format (GGUF) [21] for cross-platform compatibility. For resource-limited deployment, weight-only W4A16 quantization (e.g., AWQ [32], GPTQ [17], Q4_K [22]) has become the de facto standard, offering effective compression with minimal accuracy loss. Following established approaches, we employ Q4_K method as baseline and develop our strategy. Specifically, low-precision W^Q are quantized in IQ1_M_R4 format, an average 1.75-bit method implemented via codebook compression [24, 45]. For residuals W^R , we employ Q2_K_R4 format with an average 2.5 bits. Our experiments show that the combination of IQ1_M_R4 and Q2_K_R4 for $W^Q + W^R$ (total 4.25 bits) achieves accuracy comparable to 4.5-bit Q4_K and AWQ. Non-MoE weights are in 8-bit Q8 as they account for a negligible fraction of total weights.

Memory-Adaptive Deployment. RAPIDMoE offers adaptability for diverse platforms. At the system level, we integrate caching for low-precision W^Q . When VRAM is insufficient to accommodate W^Q , the system executes the standard RESplit path for cached W^Q on GPUs, and falls back to a CPU-centric path upon a cache miss—computing W^Q for non-critical experts and $W^Q + W^R$ for critical ones on CPUs. At the algorithmic level, the VRAM footprint is highly elastic; it can be statically tailored to specific hardware budgets by adjusting model scales or employing different quantization for W^Q . Together, this dual-level adaptability ensures RAPIDMoE maintains flexibility under arbitrary constraints.

UMIA Configuration & Frontier Search. We sample a calibration set $\mathcal{D}$ ($N = 16$) from WikiText-2 [36]. For frontier search, the candidate sets $\mathbf{C} \in (0, 1)$ and $\mathbf{A} \in (1, 1.5)$ are sampled with a resolution of $K = 100$. To prioritize foundational prefill quality, the joint objective weights are set to $\omega_p = 1.5$ and $\omega_d = 1.0$. The latency constraints are set as $S = T_{sys}(2)$.

6 Evaluation

6.1 Evaluation Setup

Models. We evaluate RAPIDMoE on SOTA large-scale MoE models with representative architectures that are mainstream

Table 3. Hardware configurations.

Platform	P1	P2
GPU	A800-80G	RTX 4090-24G
CPU (Intel Xeon)	Gold 6326 (64 core)	Platinum 8375C (128 cores)
Memory	512GB DDR4-3200	512GB DDR4-3200
DDR Bandwidth	220 GB/s	220 GB/s
PCIe	4.0 x 16	4.0 x 16

Table 4. Evaluation setups.

Configuration	Model	Platform	GPU Usage
C11	DeepSeek-R1	P1	A800×2 (160GB)
C12	DeepSeek-V3	P1	A800×2 (160GB)
C13	Qwen3-235B-A22B	P1	A800×1 (80GB)
C21	DeepSeek-R1	P2	4090×7 (168GB)

and well-received in the community: DeepSeek-V3-671B [12], DeepSeek-R1-671B [11], Qwen3-235B-A22B [50]. As shown in Table 2, DeepSeek-V3/R1 employs 256 experts per layer, with each token activating 8 routed experts plus 1 shared expert. Qwen3 adopts a more compact 128 expert configuration, routing each token to 8 experts without shared expert.

Hardware and Placement. Table 3 details evaluation platforms: (1) **P1** featuring A800-80G GPUs with 512GB CPU DDR4 memory, and (2) **P2** utilizing consumer-grade RTX 4090 GPUs with 512GB DDR4-3200. Both platforms use PCIe for GPU interconnect, with P1 and P2 comprising one and two NUMA domains, respectively. To evaluate efficiency in memory-constrained environments, we test multiple configurations listed in Table 4. For DeepSeek-R1 and V3 that require 700GB VRAM, we allocate 160GB GPU memory that sufficient for storing all low-precision experts. The smaller Qwen3 operates under an 80GB memory constraint. For multi-GPU execution, model placement is partitioned by layer: each GPU stores and executes a disjoint subset of layers. The CPU-resident weights corresponding to each GPU’s layer subset are pinned in the NUMA domain local to that GPU, minimizing cross-NUMA memory traffic.

Metrics. We evaluate system performance through prefill and decode throughput in tokens/s. For serving, we report time to first token (TTFT) for prefilling, and time per output token (TPOT) for decoding, including p50 and p99 latencies. Batch size denotes the number of requests co-executed in a model forward pass, whereas concurrency denotes the maximum number of in-flight serving requests.

Baselines. We consider the following systems as baselines:

- MoE-APEX [43], a GPU-centric system that employs adaptive precision expert offloading to reduce inference latency.
- HybriMoE [56], a CPU-centric system that utilizes dynamic scheduling and score-based caching to enhance resource utilization and workload balance.

Table 5. Accuracy evaluation under batch size 1.

Benchmark		GENERAL	MATH		CODE	
		MMLU-Pro	MATH-500	AIME-24	HumanEval	EvalPlus
DeepSeek-R1, Reasoning Model						
Q4_K_M	380GB	83.9	95.6	73.3	98.2	-
RAPIDMoE	390GB	83.8	96.2	73.3	98.2	-
Qwen3-235B-A22B, Reasoning Model						
Q4_K_M	133GB	81.0	96.4	70.0	96.3	-
RAPIDMoE	140GB	80.5	96.0	73.3	97.0	-
DeepSeek-V3, Non-Reasoning Model						
Q4_K_M	380GB	78.0	94.8	56.7	93.3	82.5
RAPIDMoE	390GB	78.2	95.2	56.7	92.7	82.6

(1) Validation (except EvalPlus) uses Evalscope [37].

(2) Reasoning models are excluded from EvalPlus as the framework lacks support for their evaluation.

- KTransformers [28], one of the most popular systems for MoE inference on memory-constrained environments with optimized CPU kernels and scheduling strategy.
- llama.cpp [22] version b6030, a lightweight C/C++ implementation for efficient LLM inference. It can statically map model layers to CPU or GPU for execution.

For a fair comparison, baselines deploy similarly sized Q4_K_M models from Unsloth [10]. Specifically for MoE-APEX that supports adaptive precision, we align its precision as INT4 (Q4_K_M) for high-precision and INT2 (IQ1_M_R4) for low-precision. We align CPU and GPU GEMM/GEMV kernel for all methods from [22], sharing identical kernels for the same quantization format and using format-specific kernels only for different dequantization requirements.

We configure all baselines with comparable memory budgets. As llama.cpp only supports layer-wise offloading, 22 of DeepSeek-R1’s 61 layers are loaded to the GPU, consuming 149GB of the available 160GB GPU memory. For others that employ expert-wise offloading, 40% of experts per layer are cached on GPUs while the remaining on CPUs. Specifically for MoE-APEX, we evaluate its two computing modes as variants: MoE-APEX-G (GPU-Centric Mode), which loads cache-missed experts via PCIe, and MoE-APEX-C (GPU-CPU Cooperative Mode), which executes missed experts on CPUs.

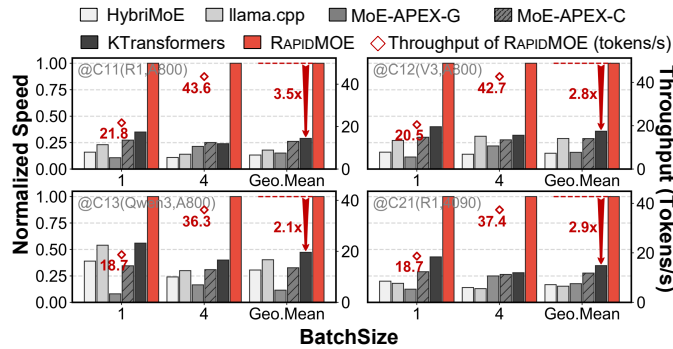
Datasets and Workloads. We construct the performance workloads from ShareGPT. Prefill experiments vary input length from 256 to 4096 tokens, while decode experiments use 512-token input prompts by default. For serving workload, we sample 1000 ShareGPT requests with an average input length of 715 tokens and cap the output length at 512 tokens. For accuracy evaluation, we benchmark on: (1) MMLU-Pro [46] (1000 randomly sampled items) for general knowledge and reasoning evaluation, (2) MATH-500 [31] and AIME-24 [8] for complex mathematical problem-solving, and (3) HumanEval [7] and EvalPlus [34] for code generation.

Table 6. Accuracy at batch size 4, with changes relative to batch size 1.

Model	MMLU-Pro	MATH-500	AIME-24	HumanEval
DeepSeek-R1	83.7 (−0.1)	96.0 (−0.2)	73.3 (0.0)	98.2 (0.0)
DeepSeek-V3	78.0 (−0.2)	94.8 (−0.4)	56.7 (0.0)	92.3 (−0.4)

Table 7. Calibration sensitivity of DeepSeek-V3 across: {ShareGPT, WikiText-2, MMLU-Pro} × {8, 16, 32} prompts.

β Variation	Max. Parameter Difference	Max. Acc. Difference
4.1–7.2%	$ \Delta\Theta , \Delta\alpha = 0.03, 0.025$ ($\leq 5/100$ grid positions)	0.0%@AIME-24 −0.4%@MMLU-Pro

**Figure 10.** Evaluation of decoding speed on DeepSeek-R1/V3 and Qwen3 models compared in different scenarios.

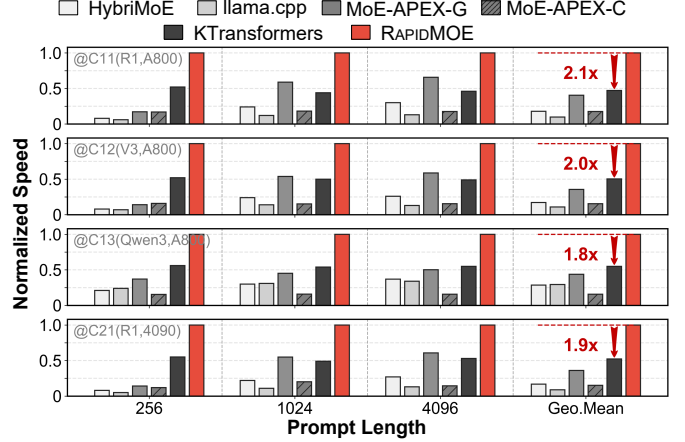
6.2 End-to-end Accuracy

As shown in Table 5, RAPIDMoE matches or exceeds Q4 accuracy. It maintains accuracy with -0.1% in general knowledge, $+0.7\%$ in mathematical reasoning, and $+0.1\%$ in code generation. This advantage holds for both reasoning models, which generate longer outputs via test-time scaling, and non-reasoning models with shorter outputs.

Robustness. RapidMoE exhibits robustness to token aggregation and calibration variations. For prefill and batched decoding, UMIA averages router scores across tokens before critical-set selection. Table 6 evaluates accuracy with batch size of 4, where token-averaged routing is applied during prefill and decode. For different calibration datasets, Table 7 shows stable accuracy, with normalized β varies by 4.1–7.2%, while the maximum differences in Θ, α vary by at most 5 of 100 grid positions. Such robustness is facilitated by UMIA’s mapping of continuous parameters and router scores to the discrete size r , allowing small perturbations to be absorbed.

6.3 End-to-end Efficiency

6.3.1 Decode Stages. Figure 10 presents the decoding performance of RAPIDMoE across four configurations (Table 4). RAPIDMoE consistently dominates all scenarios. On **P1** A800

**Figure 11.** Evaluation of prefilling speed on DeepSeek-R1/V3 and Qwen3 models compared in different scenarios.

platforms, it achieves averaged speedups of 3.5 $\times$, 2.8 $\times$, and 2.1 $\times$ for R1, V3, and Qwen3, respectively, over the SOTA KTransformers. Crucially, these gains extend to **P2**, yielding up to 2.9 $\times$ speedup with different CPU-GPU setting, demonstrating robust cross-platform efficiency.

Our evaluation exposes the fundamental bottlenecks of prior *expert-wise* offloading paradigms. GPU-centric methods (e.g., MoE-APEX-G) suffer the slowest decoding speeds due to weight transfer latency. Conversely, CPU-centric approaches (KTrans., HybriMoE) bypass PCIe bottlenecks but leave GPUs idle. Notably, while MoE-APEX attempts adaptive precision, its reliance on purely *local* gating logits fails to exploit multi-level divergence. Based on the profiling, it leads to overly preserving of 3–4 high-precision experts per layer for DeepSeek-V3, resulting in either IO bottlenecks (APEX-G) or CPU compute bottlenecks (APEX-C). In contrast, RAPIDMoE benefits from its cross-stack co-design. Algorithmically, UMIA maintains the accuracy-latency Pareto frontier by dynamically capturing a minimal critical set. System-wise, fine-grained residual offloading optimally distributes the workload and reduces GPU bubbles. Furthermore, RAPIDMoE exhibits scalability with increasing batch sizes (Table 8), benefiting from improved computation efficiency in GEMM and attention kernels at larger batches. *Long-Output* in Table 8 further evaluates AIME-24 reasoning workloads for reasoning models R1 and Qwen3, showing that RAPIDMoE’s decode throughput persists for long-output generation.

6.3.2 Prefill Stages. Figure 11 presents the prefill performance. RAPIDMoE improves prefilling throughput across varying sequence lengths, achieving 138.9 tokens/s on DeepSeek-V3 and exhibiting an average 2 $\times$ higher throughput than baselines. Notably, this speedup is less substantial than in the decode stage. This discrepancy arises as residual offloading transfers partial weight access cost to GPUs, leaving

Table 8. Decode and prefill throughput (tokens/s) for RAPID-MoE against baselines tested with 1K sequence length. *Long-Output* denotes decoding under reasoning workloads with an averaged output length at 8557.

Stage	System	Configuration, BatchSize					
		C11,1	C11,4	C12,1	C12,4	C13,1	C13,4
Decode	llama.cpp	5.4	5.7	5.3	5.8	9.7	10.0
	KTrans.	8.0	10.5	7.5	10.2	10.3	13.6
	RAPIDMoE	21.8	43.6	20.5	42.7	18.7	36.3
	<i>Long-Output</i>	18.9	37.8	–	–	17.9	34.8
Prefill	llama.cpp	17.7	18.2	17.9	18.7	60.0	61.4
	KTrans.	62.8	62.8	61.3	61.3	102.5	102.5
	RAPIDMoE	150.2	150.2	138.9	138.9	182.8	182.8

Table 9. Serving performance over 1000 ShareGPT requests at concurrency 4 and a request rate of 0.2 rps. SLO attainment is evaluated with $S_{\text{TTFT}} = 5\text{ s}$ and $S_{\text{TPOT}} = 100\text{ ms}$; Joint denotes the fraction of requests satisfying both SLOs.

Cfg.	TTFT p50/p99	TPOT p50/p99	SLO Attainment (%)		
			TTFT	TPOT	Joint
RAPIDMoE-C11	3.41s/6.21s	94ms/101ms	93.9	92.3	87.4
RAPIDMoE-C12	3.69s/6.37s	97ms/112ms	85.7	88.9	76.7

CPUs to compute only the lower-bitwidth W_R . This significantly reduces memory access overhead, yielding a greater advantage during the memory-bound decode phase. For long-context inputs (>4K), RAPIDMoE employs layer-wise overlap, transferring next-layer weights while GPUs compute the current layer. Under C11/C12, each H2D transfer costs 136 ms, while per-layer computation for an 8K input takes 116 ms, hiding most of the transfer latency. Longer inputs further increase the overlap. Across 8K, 12K, and 16K inputs, RAPIDMoE achieves 1,905 tokens/s on average, demonstrating its effectiveness for long-context prefilling.

6.3.3 Serving Performance. Table 9 reports request-level TTFT, TPOT, and SLO attainment for serving workload specified in Sec. 6.1. At 0.2 requests/s with a concurrency of 4, RAPIDMoE meets the 100ms TPOT and 5s TTFT SLOs for most requests due to its substantially higher prefill and decode throughput compared to baselines. These results demonstrate RAPIDMoE’s multi-request serving capability for low-concurrency local deployments.

6.3.4 Memory Efficiency. Table 10 details the memory breakdown for DeepSeek-V3 deployment at P1. With a comparable 149 GB peak HBM footprint, RAPIDMoE reduces peak DRAM usage to 240 GB, compared with 385 GB for KTrans. and 471 GB for MoE-APEX. The storage reduction stems from the compact RESplit representation, which stores only

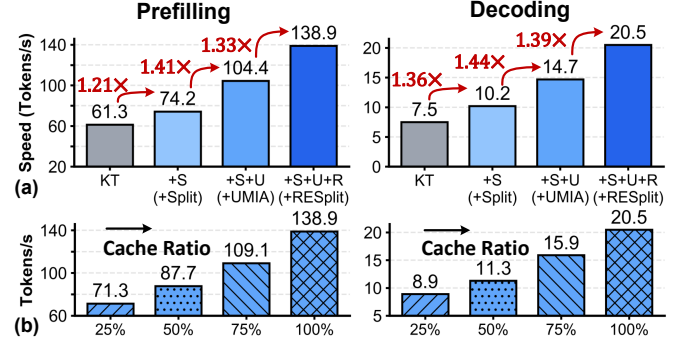


Figure 12. (a) Performance breakdown for prefilling and decoding phases on DeepSeek-V3. (b) Evaluation of different VRAM constraints, demonstrating inference speed when caching 25% to 100% of W^Q on GPUs.

Table 10. Memory breakdown at C12 configuration. HBM Other includes non-expert weights and CUDA graph/runtime buffers. Peak denotes measured peak usage.

System	HBM Breakdown (GB)			DRAM Breakdown (GB)	
	Gate/Up/Down	Other	Peak	Gate/Up/Down	Peak
MoE-APEX	44/44/44	10+6	149	157/157/157	471
KTrans.	42/42/51	10+5	150	120/120/145	385
RAPIDMoE	44/44/44	10+6	149	63/63/113	240

Q2 Gate/Up residuals and Q4 Down weights on CPUs. In contrast, KTrans. retains all Q4 expert weights, while MoE-APEX stores both low-precision (IQ1) and high-precision (Q4) copies to support adaptive precision. Thus, the RE-Split design reduces DRAM usage while delivering higher throughput as shown in Secs. 6.3.1 and 6.3.2.

6.4 Ablation Study

6.4.1 Performance Breakdown. We ablate the system to quantify the effect of each component in Figure 12 (a). We start from baseline KTransformers (KT) and incrementally integrate our proposed modules as follows:

- (1) + **S**: Incorporating split routing with a static ($r = 3$) yields a 1.2x–1.3x speedup. Statically partitioning the workload alleviates the fundamental CPU bottleneck.
- (2) + **S + U**: Dynamic arbitration provides an additional ~1.4x speedup. By navigating multi-level importance, UMIA minimizes the average CPU workload (r) without degrading model accuracy (shown in Figure 14).
- (3) + **S + U + R**: Deploying the RESplit representation and parallelism further transfers partial workload to GPUs and delivers a final speedup of up to 1.39x.

6.4.2 Efficacy of UMIA. To validate UMIA, we compare it against two static routing baselines with a fixed critical set size r : (1) *Skipping*, which discards the $k - r$ non-critical

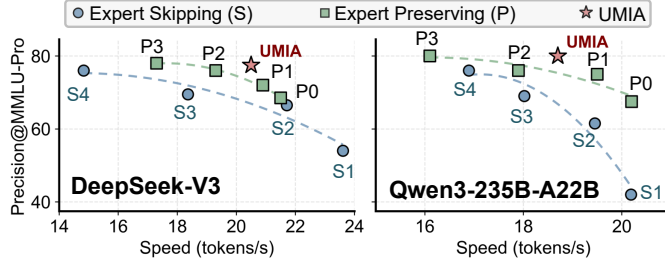


Figure 13. Accuracy-latency trade-off on (Left) DeepSeek-V3 and (Right) Qwen3-235B-A22B. Speed is evaluated using decoding throughput. S_x denotes statically skipping ($k - x$) non-critical experts, while P_x denotes preserving them in low-precision, both using local router logits for arbitration.

Table 11. Ablation of UMIA on DeepSeek-R1. All variants are calibrated on MMLU-Pro; P/D denotes prefill/decode throughput (tokens/s).

Metric	static $r = 3$	S-only r_l	T-only r_t	S+T $r_{l,t}$	L-only $g > \Theta$	UMIA
Acc.@AIME	73.3	60.0	66.7	63.3	66.7	73.3
P/D@AIME	99/16.0	157/18.2	119/17.3	150/19.9	129/16.9	141/18.9
P/D@MMLU	106/17.5	153/19.7	128/18.7	146/21.0	136/18.6	152/21.1

experts, and (2) *Preserving*, which retains them in low precision. As shown in Figure 13, *Skipping* suffers from accuracy collapse. While *Preserving* mitigates this loss, its fixed r fails to adapt to importance divergence. UMIA adjusts r based on multi-level runtime context, pushing the frontier forward with higher accuracy and throughput.

Visualization Analysis. To demystify how UMIA achieves optimality, we visualize the distribution of critical set size in Figure 14, validating the multi-level divergence in Sec. 3.1.

(1-2) **Spatial and Temporal Divergence:** UMIA adaptively assigns a larger r to shallower layers while keeping r minimal in deeper layers (spatial divergence). Temporally, prefilling exhibits a higher average r than decoding, confirming its structural preservation of foundational KV-cache quality.

(3) **Iteration-wise Drift:** Figure 14 also traces the real-time r drift for different iteration steps. The observed dynamic shifts demonstrate that a fixed static r cannot adapt to the inherent semantic drift of generated tokens over the inference process.

Modulator Ablation. We ablate UMIA’s local (L), spatial (S), and temporal (T) modulators on DeepSeek-R1, calibrating their parameters on MMLU-Pro and evaluating accuracy on AIME-24. Static variants fix r globally or along spatial (S-only) and temporal dimensions (T-only), while L-only retains only the input-dependent rule $g > \Theta$. As shown in Table 11, single-modulator variants degrade accuracy, while the static $r = 3$ preserves accuracy at the cost of lower throughput. Full UMIA recovers the accuracy while sustaining competitive throughput, demonstrating the benefit of joint adaptation.

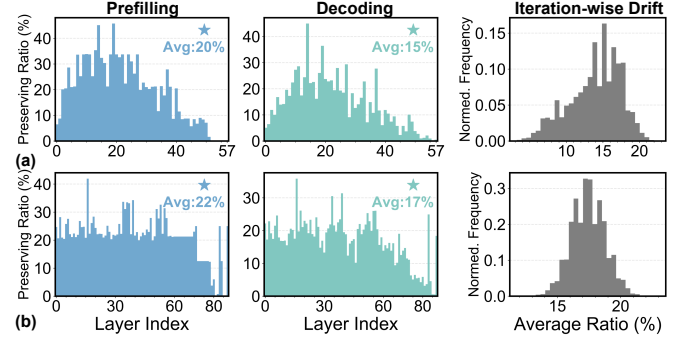


Figure 14. Visualization of preserving ratio (r/k) evaluated on MMLU-Pro for (a) DeepSeek-V3 and (b) Qwen3-235B-A22B. (Left & Middle) Averaged ratio across layers for prefill and decode phases. (Right) Distribution across iteration steps.

6.4.3 Scalability to Varying VRAM Constraints. We simulate varying VRAM constraints by capping the GPU cache capacity for low-precision experts (W^Q) in Figure 12 (b). We analyze the speed when caching 25%-100% of W^Q with footprints of 37-149GB. Results show that RAPIDMoE sustains competitive throughput even under 25% constraint, as it achieves speedups for hit experts and falls back to CPU-centric execution to handle missed experts.

Notably, while it scales to varying constraints, its performance sweet spot remains the regime where VRAM can hold the entirety of W^Q to fully unlock the benefits. It is practical for real-world deployment, since low-bit quantization shrinks the W^Q footprint of a 671B model to merely ~ 150 GB, well within the capacity of typical single-node workstations.

6.4.4 Latency Breakdown. Figure 15 presents the end-to-end latency breakdown. The results reveal two insights. First, MoE operations consistently constitute the latency bottleneck in both stages. Therefore, the optimization of MoE in RAPIDMoE significantly reduces end-to-end latency. Second, implementing batch inference during decoding enhances the efficiency of operation executed on GPUs, thereby decreasing non-MoE proportion in overall latency.

7 Discussion

Memory–Accuracy–Latency Trade-off. RAPIDMoE exposes a deployment trade-off among memory, model accuracy, and inference latency. First, RESplit prioritizes full GPU residency of W^Q , whose format determines the memory–accuracy trade-off: lower precision reduces VRAM usage but increases quantization error. CPU residual computation then recovers errors without additional VRAM consumption. Second, given a fixed W^Q format, UMIA further balances accuracy and latency by dynamically adapting the critical set at runtime. Together, RESplit and UMIA address complementary dimensions of RAPIDMoE’s deployment trade-off.

Deployment Scope. While fully GPU-resident inference is preferable when VRAM is sufficient, RAPIDMoE targets memory-constrained deployment, retaining the quantized path on GPUs while using CPU resources for selective residual correction. It enables efficient large-model inference with limited VRAM. However, this hybrid execution also defines its concurrency scope: larger batches improve GPU utilization, but provide less benefit to CPU-bound residual computation. RAPIDMoE is therefore best suited to low-concurrency, memory-constrained deployment, as higher concurrency increases TTFT and TPOT despite improving throughput.

Hardware Generality. RAPIDMoE is validated on PCIe-connected Intel CPU–NVIDIA GPU systems. However, it does not fundamentally depend on specific hardware. RESplit is built on hardware asymmetry arising from differences in compute capabilities and memory, which is expected to hold across most CPU–GPU platforms. Implementation portability depends on kernel support. Our GGUF-based GPU kernels build on llama.cpp [22], which provides CUDA, HIP, and Metal backends, while CPU kernels use common x86 extensions (AVX2) rather than Intel-specific features. Supporting other platforms requires backend-specific adaptation, with the W^Q format selected to fit the available VRAM.

The primary hardware-dependent component is the asymmetric latency model in Sec. 4.2. Higher DDR bandwidth or CPU compute throughput, including advanced AMX with compatible kernels, can accelerate CPU-bound operations. Likewise, higher GPU throughput or bandwidth benefits workloads limited by GPUs. These improvements may alter the balance between CPU and GPU. Faster CPU execution shift the transition of $R(r)$ to larger r by supporting more residual correction, whereas faster GPU execution reduces the overlap window for CPU and shifts it to smaller r . Therefore, $R(r)$ should be re-calibrated for each platform following Figure 8, and achievable gains remain hardware-dependent.

8 Related Work

Offloading Systems for MoE LLMs. The memory and latency demands of MoE LLMs have spurred the development of offloading systems for memory-constrained devices [1, 33]. GPU-centric expert-offloading systems [49, 53] execute MoE on GPUs with expert prefetching and caching to mitigate I/O overhead. Recent advances [43] optimize prefetching via layer-wise overlap and enhanced cache replacement policies, while [5] introduces novel pipelining for better throughput. However, these systems remain fundamentally limited by expert loading latency on the critical path, restricting scalability for large MoE models. Conversely, CPU-centric systems [22, 28] leverage CPU computation for expert execution through optimized kernels when experts are unavailable on GPUs, addressing latency concerns. While effective on advanced CPUs, such solutions under-utilize GPU resources, resulting in CPU bottlenecks and suboptimal throughput.

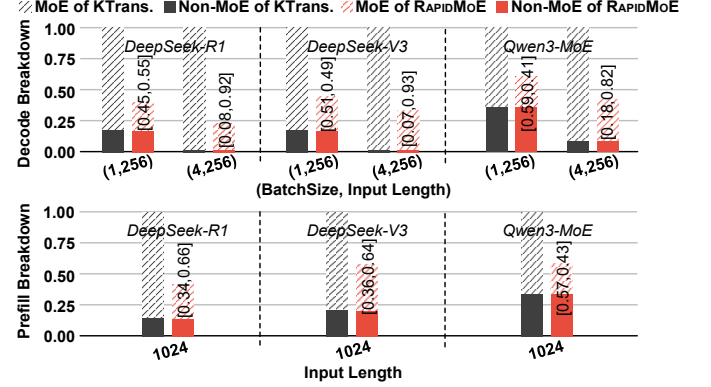


Figure 15. End-to-end latency breakdown of decode and prefill stage in KTransformers and RAPIDMoE at P1 platform.

Model Compression for MoE LLMs. Model compression has emerged as a crucial direction for efficient LLM deployment [13]. The key idea is to reduce memory footprint through methods such as pruning [16] and quantization [25]. For MoE LLMs, expert pruning [35, 51] reduces model size by systematically eliminating less important experts on calibration datasets. Quantization [25] reduces parameter precision to decrease memory footprint [14, 48]. Widely adopted quantization methods including AWQ [32], GPTQ [17], and GGUF [21] have been integrated into mainstream inference engines (e.g., vLLM [29], SGLang [55], llama.cpp [22]). Our work builds upon GGUF due to its superior cross-platform compatibility. In contrast, alternatives like AWQ and GPTQ offer limited CPU support. This choice enables consistent performance across heterogeneous devices.

9 Conclusion

We propose RAPIDMoE, an adaptive expert-residual offloading system for efficient large-scale MoE inference on heterogeneous platforms. By exploiting cross-asymmetry in algorithmic routing and hardware characteristics, RAPIDMoE employ systematic residual-split framework with a novel runtime arbitration to achieve optimal resource utilization, latency and throughput. Experiments demonstrate its practicality in real-world deployments, achieving 43 tokens/s in decode and 150 tokens/s in prefill for DeepSeek-R1-671B on 2×A800-80G GPUs. Overall, it represents a significant advancement in efficient deployment of large-scale MoE in memory-constrained environments.

Acknowledgments

We sincerely thank anonymous reviewers and our shepherd for their feedback and constructive suggestions. This work was supported by the Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China (JYB2025XDXM120), Beijing National Research Center for Information Science and Technology (BNRist).

References

- [1] Reza Yazdani Aminabadi, Samyam Rajbhandari, Minjia Zhang, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Jeff Rasley, Shaden Smith, Olatunji Ruwase, and Yuxiong He. 2022. DeepSpeed Inference: Enabling Efficient Inference of Transformer Models at Unprecedented Scale. arXiv:2207.00032 [cs.LG] <https://arxiv.org/abs/2207.00032>
- [2] Yamato Arai and Yuma Ichikawa. 2026. Quantization Error Propagation: Revisiting Layer-Wise Post-Training Quantization. arXiv:2504.09629 [cs.LG] <https://arxiv.org/abs/2504.09629>
- [3] Sikai Bai, Haoxi Li, Jie Zhang, Zicong Hong, and Song Guo. 2025. DiEP: Adaptive Mixture-of-Experts Compression through Differentiable Expert Pruning. arXiv:2509.16105 [cs.CL] <https://arxiv.org/abs/2509.16105>
- [4] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc., 1877–1901. https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfc4967418bfb8ac142f64a-Paper.pdf
- [5] Shiyi Cao, Shu Liu, Tyler Griggs, Peter Schafhalter, Xiaoxuan Liu, Ying Sheng, Joseph E. Gonzalez, Matei Zaharia, and Ion Stoica. 2025. MoE-Lightning: High-Throughput MoE Inference on Memory-constrained GPUs. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1* (Rotterdam, Netherlands) (ASPLOS '25). Association for Computing Machinery, New York, NY, USA, 715–730. doi:10.1145/3669940.3707267
- [6] Hao Mark Chen, Fuwen Tan, Alexandros Kouris, Royson Lee, Hongxiang Fan, and Stylianos I. Venieris. 2025. Progressive Mixed-Precision Decoding for Efficient LLM Inference. arXiv:2410.13461 [cs.LG] <https://arxiv.org/abs/2410.13461>
- [7] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374 [cs.LG] <https://arxiv.org/abs/2107.03374>
- [8] MAA Codeforces. [n.d.]. American Invitational Mathematics Examination-AIME 2024, 2024.
- [9] Damai Dai, Chengqi Deng, Chenggang Zhao, R. X. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y. Wu, Zhenda Xie, Y. K. Li, Panpan Huang, Fuli Luo, Chong Ruan, Zhifang Sui, and Wenfeng Liang. 2024. DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models. arXiv:2401.06066 [cs.CL] <https://arxiv.org/abs/2401.06066>
- [10] Michael Han Daniel Han and Unsloth team. 2023. *Unsloth*. <http://github.com/unslothai/unsloth>
- [11] DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948 [cs.CL] <https://arxiv.org/abs/2501.12948>
- [12] DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2025. DeepSeek-V3 Technical Report. arXiv:2412.19437 [cs.CL] <https://arxiv.org/abs/2412.19437>
- [13] Lei Deng, Guoqi Li, Song Han, Luping Shi, and Yuan Xie. 2020. Model Compression and Hardware Acceleration for Neural Networks: A Comprehensive Survey. *Proc. IEEE* 108, 4 (2020), 485–532. doi:10.1109/JPROC.2020.2976475
- [14] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. GPT3.int8(): 8-bit Matrix Multiplication for Transformers at Scale. In *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (Eds.), Vol. 35. Curran Associates, Inc., 30318–30332. https://proceedings.neurips.cc/paper_files/paper/2022/file/c3ba4962c05c49636d4c6206a97e9c8a-Paper-Conference.pdf
- [15] Artyom Eliseev and Denis Mazur. 2023. Fast Inference of Mixture-of-Experts Language Models with Offloading. arXiv:2312.17238 [cs.LG] <https://arxiv.org/abs/2312.17238>
- [16] Elias Frantar and Dan Alistarh. 2023. SparseGPT: Massive Language Models Can be Accurately Pruned in One-Shot. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 202)*, Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett (Eds.). PMLR, 10323–10337. <https://proceedings.mlr.press/v202/frantar23a.html>
- [17] Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. 2023. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. arXiv:2210.17323 [cs.LG] <https://arxiv.org/abs/2210.17323>
- [18] Elias Frantar, Roberto L. Castro, Jiale Chen, Torsten Hoefer, and Dan Alistarh. 2025. MARLIN: Mixed-Precision Auto-Regressive Parallel Inference on Large Language Models. In *Proceedings of the 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming* (Las Vegas, NV, USA) (PPoPP '25). Association for Computing Machinery, New York, NY, USA, 239–251. doi:10.1145/3710848.3710871
- [19] Othmane Friha, Mohamed Amine Ferrag, Burak Kantarci, Burak Cakmak, Arda Ozgun, and Nassira Ghoulmi-Zine. 2024. LLM-Based Edge Intelligence: A Comprehensive Survey on Architectures, Applications, Security and Trustworthiness. *IEEE Open Journal of the Communications Society* 5 (2024), 5799–5856. doi:10.1109/OJCOMS.2024.3456549
- [20] Yizhao Gao, Shuming Guo, Shijie Cao, Yuqing Xia, Yu Cheng, Lei Wang, Lingxiao Ma, Yutao Sun, Tianzhu Ye, Li Dong, Hayden Kwok-Hay So, Yu Hua, Ting Cao, Fan Yang, and Mao Yang. 2025. SeerAttention-R: Sparse Attention Adaptation for Long Reasoning. arXiv:2506.08889 [cs.LG] <https://arxiv.org/abs/2506.08889>
- [21] Georgi Gerganov. 2023. ggml: Tensor library for machine learning. GitHub repository. <https://github.com/ggerganov/ggml/blob/master/docs/gguf.md>
- [22] Georgi Gerganov. 2023. llama.cpp: Port of Facebook's LLaMA model in C/C++. GitHub repository. <https://github.com/ggerganov/llama.cpp>
- [23] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, Angela Fan, et al. 2024. The Llama 3 Herd of Models. *arXiv e-prints*, Article arXiv:2407.21783 (July 2024), arXiv:2407.21783 pages. doi:10.48550/arXiv.2407.21783 arXiv:2407.21783 [cs.AI]
- [24] ikawrakow. 2023. ikllama.cpp: llama.cpp fork with better CPU performance. GitHub repository. https://github.com/ikawrakow/ik_llama.cpp
- [25] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. 2018. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. In *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2704–2713. doi:10.1109/CVPR.2018.00286
- [26] Keisuke Kamahori, Tian Tang, Yile Gu, Kan Zhu, and Baris Kasikci. 2025. Fiddler: CPU-GPU Orchestration for Fast Inference of Mixture-of-Experts Models. arXiv:2402.07033 [cs.LG] <https://arxiv.org/abs/2402.07033>
- [27] Takeshi Kojima, Shixiang (Shane) Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large Language Models are Zero-Shot Reasoners. In *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (Eds.), Vol. 35. Curran Associates, Inc., 22199–22213. https://proceedings.neurips.cc/paper_files/paper/2022/file/8bb0d291acd4acf06ef112099c16f326-Paper-Conference.pdf

- [28] KVCache-AI. 2024. Ktransformers: A flexible framework for experiencing cutting-edge llm inference optimizations. GitHub repository. <https://github.com/kvcache-ai/ktransformers>
- [29] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (Koblenz, Germany) (SOSP '23)*. Association for Computing Machinery, New York, NY, USA, 611–626. doi:10.1145/3600006.3613165
- [30] Dmitry Lepikhin, Hyoungho Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. 2020. GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding. arXiv:2006.16668 [cs.CL] <https://arxiv.org/abs/2006.16668>
- [31] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2023. Let's Verify Step by Step. arXiv:2305.20050 [cs.LG] <https://arxiv.org/abs/2305.20050>
- [32] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration. In *Proceedings of Machine Learning and Systems*, P. Gibbons, G. Pekhimenko, and C. De Sa (Eds.), Vol. 6. 87–100. https://proceedings.mlsys.org/paper_files/paper/2024/file/42a452cbafa9dd64e9ba4aa95cc1ef21-Paper-Conference.pdf
- [33] Jiacheng Liu, Peng Tang, Wenfeng Wang, Yuhang Ren, Xiaofeng Hou, Pheng-Ann Heng, Minyi Guo, and Chao Li. 2025. A Survey on Inference Optimization Techniques for Mixture of Experts Models. arXiv:2412.14219 [cs.LG] <https://arxiv.org/abs/2412.14219>
- [34] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and LINGMING ZHANG. 2023. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. In *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., 21558–21572. https://proceedings.neurips.cc/paper_files/paper/2023/file/43e9d647ccd3e4b7b5baab53f0368686-Paper-Conference.pdf
- [35] Xudong Lu, Qi Liu, Yuhui Xu, Aojun Zhou, Siyuan Huang, Bo Zhang, Junchi Yan, and Hongsheng Li. 2024. Not All Experts are Equal: Efficient Expert Pruning and Skipping for Mixture-of-Experts Large Language Models. arXiv:2402.14800 [cs.CL] <https://arxiv.org/abs/2402.14800>
- [36] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2016. Pointer Sentinel Mixture Models. arXiv:1609.07843 [cs.CL] <https://arxiv.org/abs/1609.07843>
- [37] Modelscope. 2024. Evalscope: Evaluation Framework for Large Models. GitHub repository. <https://github.com/modelscope/evalscope>
- [38] OpenAI, :, Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, et al. 2024. OpenAI o1 System Card. arXiv:2412.16720 [cs.AI] <https://arxiv.org/abs/2412.16720>
- [39] OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, et al. 2024. GPT-4 Technical Report. arXiv:2303.08774 [cs.CL] <https://arxiv.org/abs/2303.08774>
- [40] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. arXiv:1701.06538 [cs.LG] <https://arxiv.org/abs/1701.06538>
- [41] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Re, Ion Stoica, and Ce Zhang. 2023. FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 202)*, Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett (Eds.). PMLR, 31094–31116. <https://proceedings.mlr.press/v202/sheng23a.html>
- [42] Andrii Skliar, Ties van Rozendaal, Romain Lepert, Todor Boinovski, Mart van Baalen, Markus Nagel, Paul Whatmough, and Babak Ehteshami Bejnordi. 2025. Mixture of Cache-Conditional Experts for Efficient Mobile Device Inference. arXiv:2412.00099 [cs.LG] <https://arxiv.org/abs/2412.00099>
- [43] Peng Tang, Jiacheng Liu, Xiaofeng Hou, Yifei Pu, Jing Wang, Pheng-Ann Heng, Chao Li, and Minyi Guo. 2026. MoE-APEX: An Efficient MoE Inference System with Adaptive Precision Expert Offloading. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (USA) (ASPLOS '26)*. Association for Computing Machinery, New York, NY, USA, 1185–1200. doi:10.1145/3779212.3790187
- [44] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and Efficient Foundation Language Models. arXiv:2302.13971 [cs.CL] <https://arxiv.org/abs/2302.13971>
- [45] Albert Tseng, Jerry Chee, Qingyao Sun, Volodymyr Kuleshov, and Christopher De Sa. 2024. QuIP#: Even Better LLM Quantization with Hadamard Incoherence and Lattice Codebooks. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*, Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp (Eds.). PMLR, 48630–48656. <https://proceedings.mlr.press/v235/tseng24a.html>
- [46] Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyang Jiang, Tianle Li, Max Ku, Kai Wang, Alex Zhuang, Rongqi Fan, Xiang Yue, and Wenhui Chen. 2024. MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark. In *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (Eds.), Vol. 37. Curran Associates, Inc., 95266–95290. https://proceedings.neurips.cc/paper_files/paper/2024/file/ad236edc564f3e3156e1b2feafb99a24-Paper-Datasets_and_Benchmarks_Track.pdf
- [47] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, brian ichter, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (Eds.), Vol. 35. Curran Associates, Inc., 24824–24837. https://proceedings.neurips.cc/paper_files/paper/2022/file/9d5609613524ecf4f15af0f7b31abca4-Paper-Conference.pdf
- [48] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2023. SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 202)*, Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett (Eds.). PMLR, 38087–38099. <https://proceedings.mlr.press/v202/xiao23c.html>
- [49] Leyang Xue, Yao Fu, Zhan Lu, Luo Mai, and Mahesh Marina. 2025. MoE-Infinity: Efficient MoE Inference on Personal Machines with Sparsity-Aware Expert Cache. arXiv:2401.14361 [cs.LG] <https://arxiv.org/abs/2401.14361>

- [50] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 Technical Report. arXiv:2505.09388 [cs.CL] <https://arxiv.org/abs/2505.09388>
- [51] Cheng Yang, Yang Sui, Jinqi Xiao, Lingyi Huang, Yu Gong, Yuanlin Duan, Wenqi Jia, Miao Yin, Yu Cheng, and Bo Yuan. 2024. MoE-I²: Compressing Mixture of Experts Models through Inter-Expert Pruning and Intra-Expert Low-Rank Decomposition. arXiv:2411.01016 [cs.LG] <https://arxiv.org/abs/2411.01016>
- [52] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. 2025. FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving. *arXiv e-prints*, Article arXiv:2501.01005 (Jan. 2025), arXiv:2501.01005 pages. doi:10.48550/arXiv.2501.01005 arXiv:2501.01005 [cs.DC]
- [53] Rongjie Yi, Liwei Guo, Shiyun Wei, Ao Zhou, Shangguang Wang, and Mengwei Xu. 2025. EdgeMoE: Empowering Sparse Large Language Models on Mobile Devices. arXiv:2308.14352 [cs.LG] <https://arxiv.org/abs/2308.14352>
- [54] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, Yifan Du, Chen Yang, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Peiyu Liu, Jian-Yun Nie, and Ji-Rong Wen. 2025. A Survey of Large Language Models. arXiv:2303.18223 [cs.CL] <https://arxiv.org/abs/2303.18223>
- [55] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (Eds.), Vol. 37. Curran Associates, Inc., 62557–62583. https://proceedings.neurips.cc/paper_files/paper/2024/file/724be4472168f31ba1c9ac630f15dec8-Paper-Conference.pdf
- [56] Shuzhang Zhong, Yanfan Sun, Ling Liang, Runsheng Wang, Ru Huang, and Meng Li. 2025. HybriMoE: Hybrid CPU-GPU Scheduling and Cache Management for Efficient MoE Inference. *arXiv preprint arXiv:2504.05897* (2025).